DuocUC | ONLINE



Desarrollo Backend III

Informe técnico

Cristian Javier Torres Martínez

11-10-2026

Índice

1. Introducción 3

2. Objetivos 3

3. Arquitectura de la solución 3

3.1 Diagrama de arquitectura.....4

3.2 Componentes principales.....5

4. Implementación de microservicios y patrón BFF 5

4.1 Gestión de cuentas y operaciones bancarias.....6

5. Seguridad y resiliencia 7

5.1 Seguridad 7

5.2 Comunicación y tolerancia a fallos.....7

6. Integración con Apache Kafka y auditoría 8

8. Persistencia y despliegue.....10

10. Desafíos y soluciones 11

11. Mejoras futuras.....11

12. Conclusión 12

13. Referencias.....12

1. Introducción

El Banco XYZ requiere modernizar su plataforma bancaria para superar las limitaciones de su sistema legacy, mejorar la mantenibilidad y facilitar la integración de nuevas funcionalidades. Para abordar esta necesidad, se desarrolló una solución basada en microservicios con Java, Spring Boot y Spring Cloud. La arquitectura separa las responsabilidades bancarias, incorpora interfaces BFF para distintos canales, mecanismos de seguridad con OAuth 2.0 y JWT, comunicación asíncrona mediante Apache Kafka y procesamiento de archivos financieros con Spring Batch.

2. Objetivos

Objetivo general

Desarrollar una arquitectura backend distribuida que permita gestionar operaciones bancarias mediante servicios independientes, seguros y preparados para su despliegue con contenedores.

Objetivos específicos

- Separar las funcionalidades bancarias en microservicios.
- Implementar BFF para los canales web, móvil y cajeros automáticos.
- Centralizar la configuración y facilitar el descubrimiento de servicios.
- Incorporar autenticación, autorización y mecanismos de resiliencia.
- Utilizar Kafka para intercambiar eventos de cuentas y transacciones.
- Migrar los procesos de archivos financieros a Spring Batch.
- Facilitar el despliegue y la administración mediante Docker Compose.

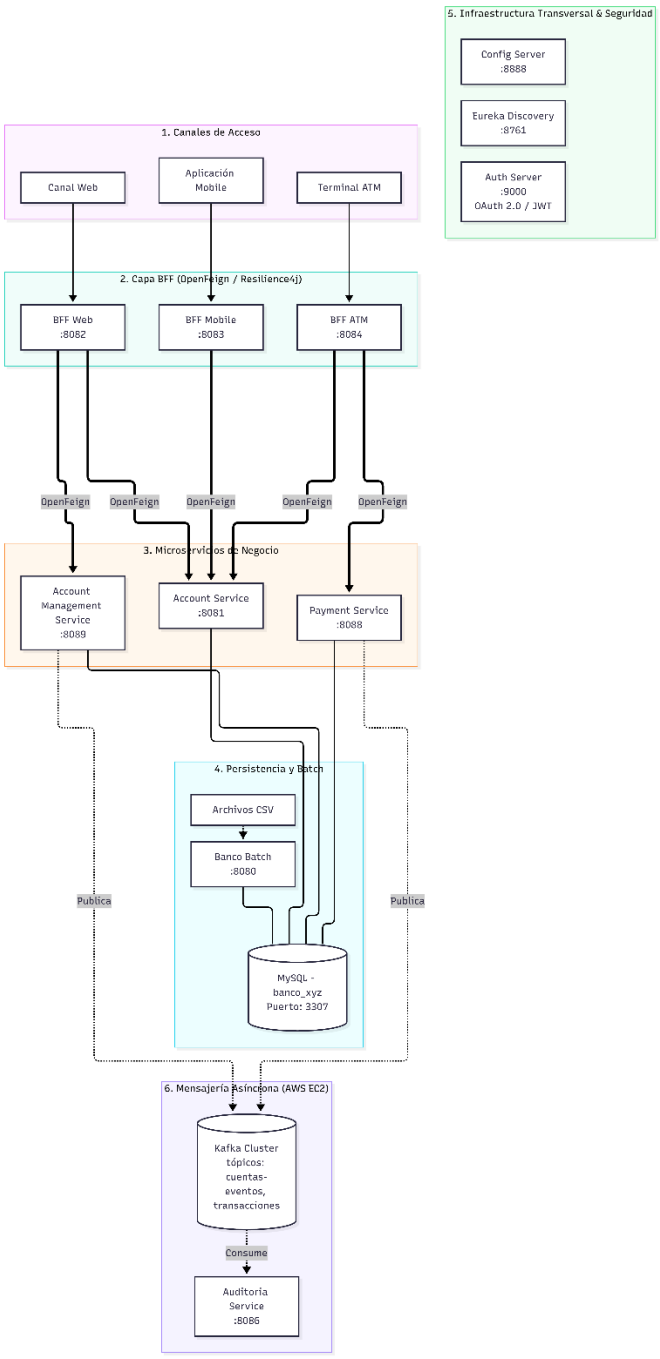
3. Arquitectura de la solución

La solución se organiza en microservicios que colaboran mediante comunicación HTTP y eventos asíncronos. Durante la refactorización, el antiguo backend-core se eliminó y sus

responsabilidades se distribuyeron entre account-service, account-management-service y payment-service.

La infraestructura de aplicación se ejecuta mediante Docker Compose local, mientras que el clúster Kafka está desplegado de forma independiente en Amazon EC2.

3.1 Diagrama de arquitectura



3.2 Componentes principales

Componente	Responsabilidad
BFF Web	Adaptar las respuestas y operaciones para el canal web.
BFF Mobile	Entregar información adaptada al cliente móvil.
BFF ATM	Coordinar consultas y operaciones de cajeros automáticos.
Account Service	Consultar cuentas y transacciones.
Account Management Service	Crear, consultar, actualizar y eliminar cuentas.
Payment Service	Ejecutar retiros y registrar operaciones de pago implementadas.
Auth Server	Gestionar la emisión de tokens de acceso.
Config Server	Centralizar la configuración de los servicios.
Discovery Server	Registrar y localizar servicios mediante Eureka.
Auditoria Service	Consumir y registrar eventos recibidos desde Kafka.
Banco Batch	Procesar archivos financieros mediante Spring Batch.
MySQL	Persistir las cuentas, transacciones y resultados configurados.
Kafka	Transportar eventos entre productores y consumidores.

4. Implementación de microservicios y patrón BFF

Se implementaron tres BFF independientes, cada uno orientado a las necesidades de un canal. Estos reciben las solicitudes de los clientes y se comunican con los microservicios correspondientes mediante clientes OpenFeign.
El BFF Web utiliza AccountServiceClient y AccountManagementClient; el BFF Mobile utiliza AccountServiceClient; y el BFF ATM utiliza AccountServiceClient y PaymentServiceClient.

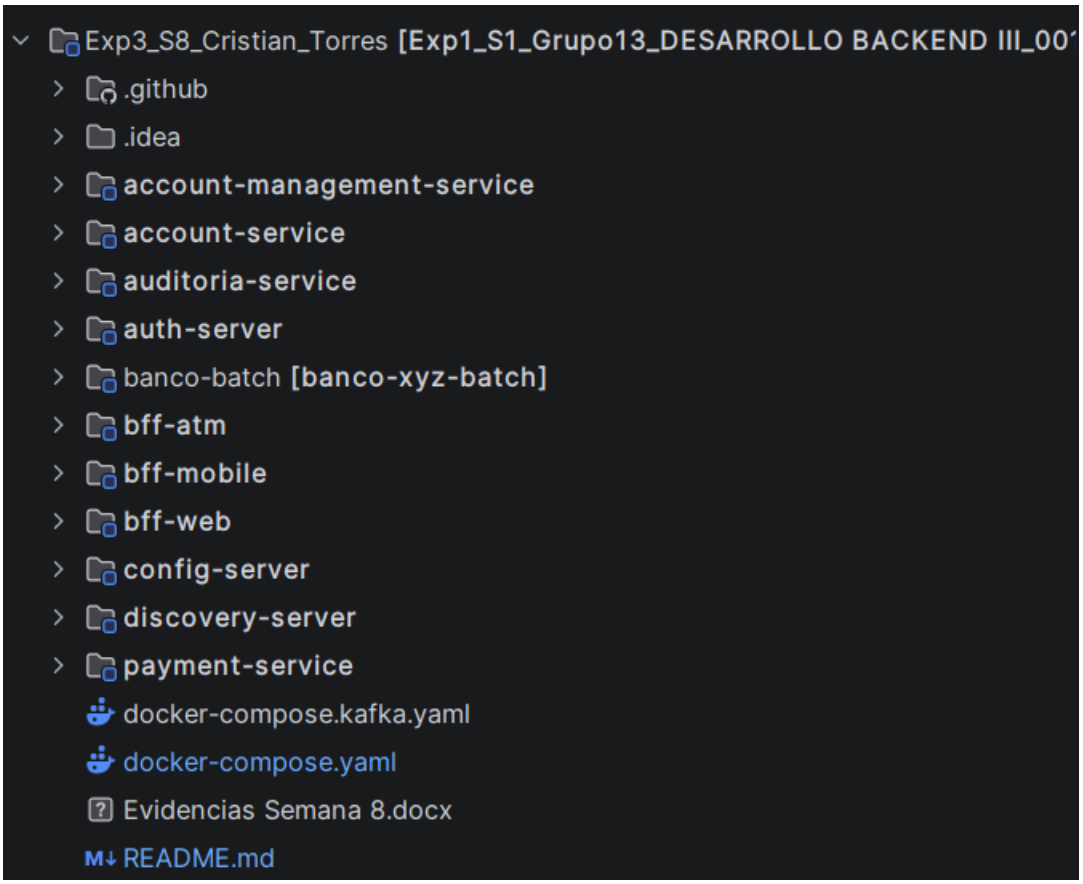
La separación permite adaptar las respuestas por canal y evita concentrar todas las responsabilidades en un único backend.

4.1 Gestión de cuentas y operaciones bancarias

La lógica anteriormente concentrada en backend-core se distribuyó en tres servicios:

- account-service: consulta de cuentas y movimientos.
- account-management-service: administración del ciclo de vida de las cuentas.
- payment-service: procesamiento de retiros y publicación de eventos de transacciones.

Cada servicio mantiene sus controladores, DTOs, lógica de negocio, repositorios y configuración de seguridad según sus responsabilidades.



5. Seguridad y resiliencia

5.1 Seguridad

Se incorporó un servidor de autorización con Spring Authorization Server, OAuth 2.0 y tokens JWT. Los BFF y los tres microservicios de negocio mencionados cuentan con configuración de seguridad propia.

El flujo general consiste en obtener un token desde el Auth Server y utilizarlo en las solicitudes protegidas mediante el encabezado Authorization: Bearer. Cada servicio aplica las reglas de acceso definidas en su configuración.

La seguridad del transporte mediante HTTPS depende de la configuración de TLS del entorno de despliegue; no se considera habilitada únicamente por utilizar JWT.

5.2 Comunicación y tolerancia a fallos

OpenFeign facilita las llamadas HTTP entre servicios, mientras que Eureka permite localizar los servicios registrados. Spring Cloud LoadBalancer participa en el balanceo de solicitudes cuando se encuentra configurado para ello.

Se incorporaron mecanismos de resiliencia con Resilience4j en los BFF, incluyendo estrategias configuradas para gestionar fallos temporales y ofrecer respuestas controladas ante problemas de disponibilidad.

DS Replicas

localhost

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
ACCOUNT-MANAGEMENT-SERVICE	n/a (1)	(1)	UP (1) - 6a7830b89e67:account-management-service:8089
ACCOUNT-SERVICE	n/a (1)	(1)	UP (1) - 978ccd1b249f:account-service:8081
AUDITORIA-SERVICE	n/a (1)	(1)	UP (1) - e33532484af0:auditoria-service:8086
BFF-ATM	n/a (1)	(1)	UP (1) - 00dd1ee64b9a:bff-atm:8084
BFF-MOBILE	n/a (1)	(1)	UP (1) - 6913cdd695f2:bff-mobile:8083
BFF-WEB	n/a (1)	(1)	UP (1) - bdd26e02c75a:bff-web:8082
PAYMENT-SERVICE	n/a (1)	(1)	UP (1) - cfa34ece9d47:payment-service:8088

6. Integración con Apache Kafka y auditoría

Kafka permite intercambiar eventos sin que el productor tenga que esperar a que el servicio de auditoría termine de procesarlos.

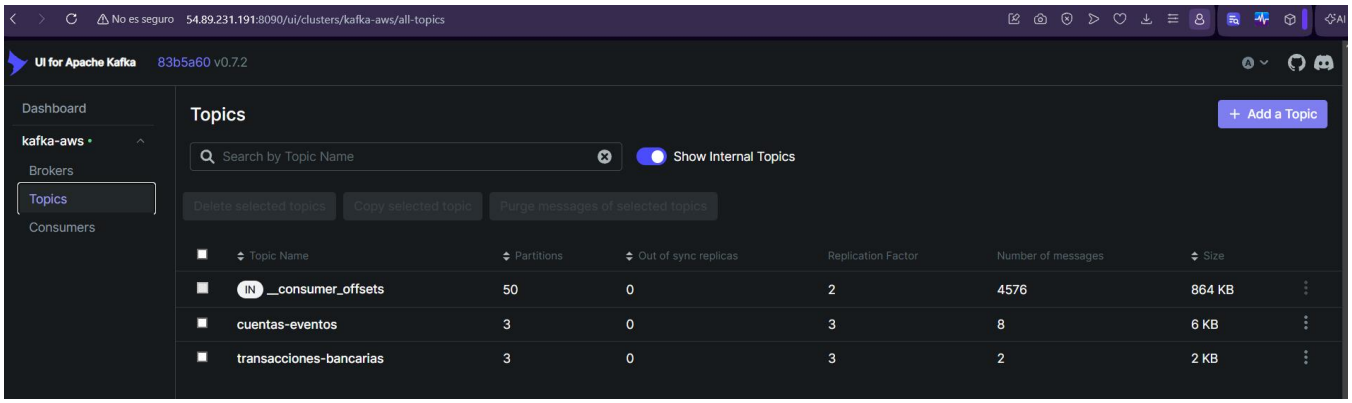
Se configuraron dos tópicos:

- cuentas-eventos: recibe eventos publicados por account-management-service.
- transacciones-bancarias: recibe eventos publicados por payment-service.

El auditoria-service consume ambos tópicos mediante CuentaConsumer y TransaccionConsumer, utilizando configuraciones de deserialización independientes para cada tipo de evento.

La infraestructura Kafka se ejecuta en Amazon EC2, separada de los microservicios locales.

En la implementación actual, los eventos recibidos se registran en los logs de auditoría. No se considera persistencia histórica de auditoría en MySQL, ya que no forma parte de la estructura implementada.



7. Migración de procesos batch con Spring Batch

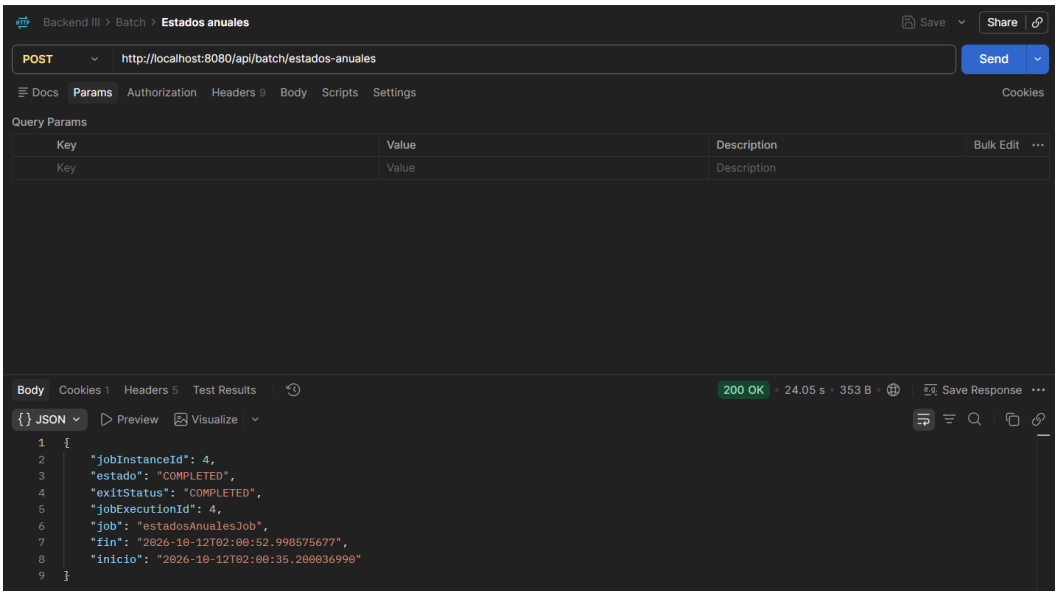
El módulo banco-batch procesa archivos CSV con información financiera. Los procesos se estructuran mediante jobs y steps, con componentes para lectura, validación, transformación y escritura de datos.

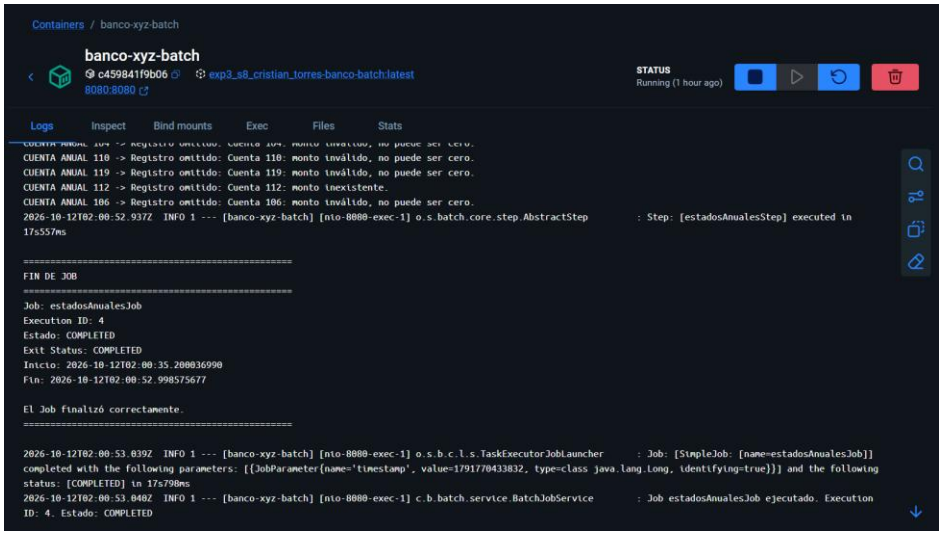
Los archivos de entrada son:

- movimientos_financieros_diarios.csv

- intereses_trimestrales.csv
- estados_financieros_anuales.csv

Entre las clases relevantes se encuentran TransaccionesJobConfig, InteresesJobConfig, EstadosAnualesJobConfig y ProcesoBatchCompletoJobConfig. También se utilizan procesadores, writers, listeners, repositorios y un tasklet para el resumen de anomalías.
El tratamiento de errores se apoya en validaciones y componentes de manejo de registros omitidos. Los reintentos deben aplicarse a fallos transitorios según las políticas configuradas, mientras que los errores de datos requieren validación o tratamiento específico.
El uso de procesamiento por lotes permite organizar la ejecución y controlar la escritura de resultados en MySQL.



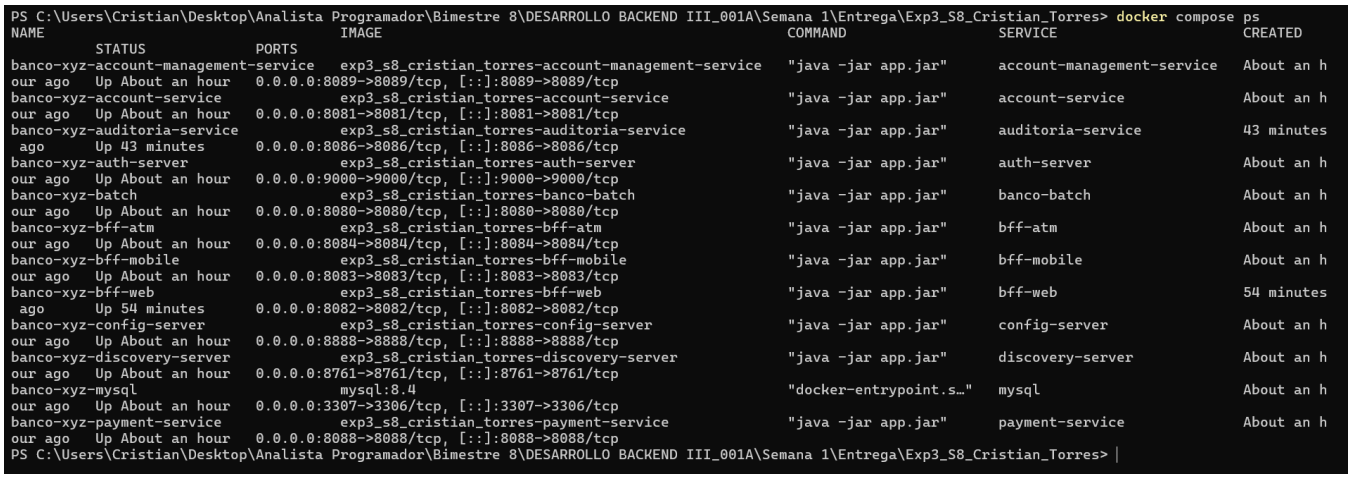


8. Persistencia y despliegue

MySQL 8.4 se utiliza como base de datos relacional. Los microservicios acceden a los datos mediante sus repositorios, y el módulo batch almacena los resultados definidos por su configuración.

Docker Compose permite iniciar y administrar los contenedores de la aplicación. Los servicios se comunican mediante los nombres DNS internos de Compose, mientras que Kafka utiliza las direcciones configuradas para el clúster de EC2.

La base de datos cuenta con un volumen persistente para conservar los datos entre reinicios de los contenedores.



```
PS C:\Users\Cristian\Desktop\Analista Programador\Bimestre 8\DESARROLLO BACKEND III_001A\Semana 1\Entrega\Exp3_S8_Cristian_Torres> docker compose ps
NAME                                IMAGE                                                COMMAND                  SERVICE                      CREATED
        STATUS              PORTS
banco-xyz-account-management-service   exp3_s8_cristian_torres-account-management-service   "java -jar app.jar"      account-management-service   About an h
our ago   Up About an hour   0.0.0.0:8089->8089/tcp, [::]:8089->8089/tcp
banco-xyz-account-service              exp3_s8_cristian_torres-account-service              "java -jar app.jar"      account-service              About an h
our ago   Up About an hour   0.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp
banco-xyz-auditoria-service            exp3_s8_cristian_torres-auditoria-service            "java -jar app.jar"      auditoria-service            43 minutes
 ago      Up 43 minutes      0.0.0.0:8086->8086/tcp, [::]:8086->8086/tcp
banco-xyz-auth-server                  exp3_s8_cristian_torres-auth-server                  "java -jar app.jar"      auth-server                  About an h
our ago   Up About an hour   0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp
banco-xyz-batch                        exp3_s8_cristian_torres-banco-batch                  "java -jar app.jar"      banco-batch                  About an h
our ago   Up About an hour   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
banco-xyz-bff-atm                      exp3_s8_cristian_torres-bff-atm                      "java -jar app.jar"      bff-atm                      About an h
our ago   Up About an hour   0.0.0.0:8084->8084/tcp, [::]:8084->8084/tcp
banco-xyz-bff-mobile                   exp3_s8_cristian_torres-bff-mobile                   "java -jar app.jar"      bff-mobile                   About an h
our ago   Up About an hour   0.0.0.0:8083->8083/tcp, [::]:8083->8083/tcp
banco-xyz-bff-web                      exp3_s8_cristian_torres-bff-web                      "java -jar app.jar"      bff-web                      54 minutes
 ago      Up 54 minutes      0.0.0.0:8082->8082/tcp, [::]:8082->8082/tcp
banco-xyz-config-server                exp3_s8_cristian_torres-config-server                "java -jar app.jar"      config-server                About an h
our ago   Up About an hour   0.0.0.0:8888->8888/tcp, [::]:8888->8888/tcp
banco-xyz-discovery-server             exp3_s8_cristian_torres-discovery-server             "java -jar app.jar"      discovery-server             About an h
our ago   Up About an hour   0.0.0.0:8761->8761/tcp, [::]:8761->8761/tcp
banco-xyz-mysql                        mysql:8.4                                            "docker-entrypoint.s…"   mysql                        About an h
our ago   Up About an hour   0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp
banco-xyz-payment-service              exp3_s8_cristian_torres-payment-service              "java -jar app.jar"      payment-service              About an h
our ago   Up About an hour   0.0.0.0:8088->8088/tcp, [::]:8088->8088/tcp
PS C:\Users\Cristian\Desktop\Analista Programador\Bimestre 8\DESARROLLO BACKEND III_001A\Semana 1\Entrega\Exp3_S8_Cristian_Torres>
```

9. Pruebas y resultados

Durante la validación se comprobó la integración de los componentes principales de la solución:

- Registro y descubrimiento de servicios mediante Eureka.
- Comunicación entre BFF y microservicios.
- Ejecución de operaciones bancarias implementadas.
- Publicación y consumo de eventos de cuentas y transacciones mediante Kafka.
- Ejecución de los procesos batch y escritura de resultados en MySQL.
- Inicio y administración de los servicios mediante Docker Compose.

Las pruebas de Kafka confirmaron la recepción de eventos de cuentas y de transacciones en los consumidores correspondientes. Los resultados de las demás pruebas deben respaldarse con las ejecuciones verificadas del proyecto.
La solución reemplaza el enfoque monolítico por responsabilidades distribuidas, facilita la evolución independiente de los componentes y añade mecanismos de seguridad, comunicación asíncrona y procesamiento batch.

10. Desafíos y soluciones

Uno de los principales desafíos fue dividir las responsabilidades del backend original sin perder la integración entre consultas, administración de cuentas y operaciones bancarias. Para resolverlo, se separaron las funcionalidades en tres microservicios y se actualizaron los clientes de los BFF.
Otro desafío fue configurar Kafka para manejar dos tipos de eventos. Se solucionó utilizando fábricas de consumidores independientes para los eventos de cuentas y transacciones.
También fue necesario coordinar las configuraciones de red y conexión entre los contenedores locales, MySQL, Eureka y Kafka en EC2. Se abordó mediante variables de entorno y configuraciones específicas para cada entorno.

11. Mejoras futuras

Como próximos pasos se propone:

- Incorporar persistencia histórica de auditoría y consultas de eventos.
- Ampliar las pruebas automatizadas unitarias y de integración.
- Reforzar la observabilidad con métricas, trazas y alertas centralizadas.
- Validar las políticas de reintentos e idempotencia en operaciones bancarias.
- Configurar y verificar HTTPS para los entornos que lo requieran.
- Automatizar la construcción, validación y despliegue mediante CI/CD.
- Realizar pruebas de carga y recuperación ante fallos.

Estas mejoras permitirían aumentar la trazabilidad, la capacidad de diagnóstico y la preparación del sistema para un entorno productivo.

12. Conclusión

El proyecto Banco XYZ implementa una arquitectura backend basada en microservicios que distribuye las responsabilidades de consulta, administración y operación bancaria. La solución integra BFF para web, móvil y cajeros automáticos, seguridad mediante OAuth 2.0 y JWT, descubrimiento de servicios con Eureka, resiliencia, eventos asíncronos con Kafka y procesamiento de archivos financieros con Spring Batch.
La contenedorización con Docker Compose facilita la ejecución del entorno y la separación de Kafka en Amazon EC2 permite mantener independiente la infraestructura de mensajería.
El trabajo constituye una base para continuar la modernización del sistema bancario, incorporando progresivamente mejoras de observabilidad, pruebas, seguridad del transporte y persistencia de auditoría.

13. Referencias

- Spring Batch. Documentación oficial: https://docs.spring.io/spring-batch/reference/
- Spring Security. OAuth 2.0 Resource Server con JWT: https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html
- Spring for Apache Kafka. Documentación oficial: https://docs.spring.io/spring-kafka/reference/
- Spring Cloud. Documentación oficial: https://spring.io/projects/spring-cloud
- Docker Compose. Documentación oficial: https://docs.docker.com/compose/
- Repositorio de datos legacy de la actividad: https://github.com/KariVillagran/fin_legacy_data
- Repositorio del proyecto Banco XYZ: [Insertar URL del repositorio GitHub del proyecto]



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC
La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.